



SCHOOL OF COMPUTER SCIENCES
UNIVERSITI SAINS MALAYSIA

CPT212 : Design & Analysis Algorithm
Semester II, Academic Session 2025/2026

ASSIGNMENT 2 (15%)

Lecturer: Dr. Hazrina Yusof Hamdani

STUDENT NAME	MATRIC NUMBER
HARIS ZAKUWAN BIN SA-UDI	23302870
IRDINA ILMUNA BINTI MOHD NASIR	22306872
TUAN NURLIYANA BINTI TUAN AB RAHMAN	22306920

Github Repository : <https://github.com/Skywalkz1/CPT212-Assignment2-.git>

Date of submission:

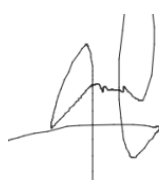
14th June 2026

**CPT212/CPT212W – Design & Analysis of
Algorithms Semester 2, Academic Year 2025/2026**

Assignment 2

Students' declaration:

We declare that we understand what is meant by plagiarism. This assignment is all our own work and we have acknowledged any use of the published or unpublished works of other people. We hold a copy of this assignment. We can produce if the original is lost or damaged.

Student Name & Student ID	Signature	Task Distribution
HARIS ZAKUWAN BIN SA-UDI (23302870)		Part A: Boyer-Moore Algorithm
IRDINA ILMUNA BINTI MOHD NASIR (22306872)		Part B: Variations of the Boyer-Moore Algorithm
TUAN NURLIYANA BINTI TUAN AB RAHMAN (22306920)		Part C: Comparison with the KMP Algorithm

Lecturer has, and may exercise, the right NOT TO MARK this assignment if the above declaration has NOT BEEN SIGNED and if the above declaration is FOUND TO BE FALSE, appropriate action will be taken which would lead to ZERO marks being awarded for this assignment

This **GROUP** assignment shall contribute **15%** of the overall evaluation.

Table of Contents

1.0 Part A : Boyer-Moore Algorithm	3
1.1 The Concept of String Matching and Its Applications	3
1.1.1 Applications in Computing and Real-World Systems	3
1.2 Boyer-Moore Implementation, Output Analysis and Discussion	4
1.2.1 How Boyer-Moore Works	4
1.2.1 Python Implementation	5
1.2.2 Output Analysis	8
1.2.3 Time and Space Complexity Analysis	10
1.3 Strengths and Weaknesses of Boyer-Moore	10
1.3.1 Strengths of Boyer-Moore	10
1.3.2 Weaknesses of Boyer Moore	11
2.0 Part B : Variations of the Boyer-Moore Algorithm	12
2.1 Introduction to Algorithm Variations	12
2.2 Variation 1: Boyer-Moore-Horspool Algorithm	13
2.2.1 Key Differences Between Boyer-Moore-Horspool and Boyer-Moore Algorithm	13
2.2.2 Pseudocode & Complexity Analysis	14
2.2.3 Strengths and Weaknesses	14
2.3 Variation 2: Turbo Boyer Moore Algorithm	15
2.3.1 Key Differences Between Turbo Boyer-Moore Algorithm and Boyer-Moore Algorithm	15
2.3.2 Pseudocode & Complexity Analysis	16
2.3.3 Strengths & Weaknesses	17
2.4 Variation 3: Sunday Algorithm	18
2.4.1 Key Differences Between Sunday Algorithm and Boyer-Moore Algorithm	18
2.4.2 Pseudocode & Complexity Analysis	19
2.5 Comparative Analysis	20
2.6 Performance Considerations	21
3.0 Part C : Comparison with the KMP Algorithm	22
3.1 Introduction	22
3.1.1 Primary Mechanism: The Prefix Table (LPS Array)	22
3.1.2 Illustration with an Example	23
3.1.3 KMP Algorithm Pseudocode	23
3.2 Comparison of the Boyer-Moore and KMP Algorithms	25
3.2.1 Comparison Summary	25
3.2.2 Pseudocode Illustration of Complexity Differences	25
3.2.3 Summary of Comparison	27
3.3 Scenarios: Algorithm Suitability	28
4.0 References	30

1.0 Part A : Boyer-Moore Algorithm

1.1 The Concept of String Matching and Its Applications

String matching (also called string searching) is a fundamental problem in computer science that involves locating one or more occurrences of a pattern string within a larger text string. Formally, given a text T of length n and a pattern P of length m (where $m \leq n$), the objective is to find all positions i such that:

$$T[i], T[i + 1], \dots, T[i + m - 1] = P[0], P[1], \dots, P[m - 1]$$

The brute-force approach aligns the pattern at every possible position $s \in \{0, 1, \dots, n - m\}$ and compares character by character. Hence worst case time complexity is $O(nm)$. This is computationally expensive for huge texts or extended patterns. Advanced algorithms such as Boyer-Moore lower this cost by using structural aspects of the pattern in a preprocessing phase to safely skip significant chunks of the text in the actual search phase. (Boyer & Moore, 1977)

String matching algorithms generally work in two phases:

Phase	Description
Preprocessing	Analyse the pattern P to build auxiliary data structures (shift tables, failure functions, hash values). Time cost: $O(m)$.
Searching	Slide the pattern over the text and use the preprocessed structures to skip unnecessary comparisons. Time cost varies by algorithm.

1.1.1 Applications in Computing and Real-World Systems

String matching is not a theoretical exercise, it underpins a wide range of critical systems:

1. Text Editors and Word Processors

The "Find" and "Find & Replace" functions in editors such as Microsoft Word, VS Code, and Vim rely directly on string-searching algorithms. Efficient matching ensures that even searches across documents with hundreds of thousands of characters long return results instantaneously.

2. Search Engines and Information Retrieval

Web search engines (Google, Bing) use sophisticated string matching at the inverted-index stage to locate query terms within document tokens. Exact-match pattern searching is the foundation upon which more complex ranking and relevance scoring algorithms are built (Manning, Raghavan & Schütze, 2008).

3. Bioinformatics and DNA Sequencing

Genome databases store sequences of billions of nucleotide characters (A, C, G, T). Researchers use string matching to locate gene subsequences, primer sites, restriction enzyme cut sites, and mutation sites.

4. Network Intrusion Detection Systems (IDS)

Tools such as Snort and Suricata perform deep packet inspection by matching packet payloads against a library of known attack signatures. The Boyer-Moore algorithm is employed here precisely because of its sub-linear average-case performance, enabling real-time analysis of high-throughput network traffic.

5. Compilers and Lexical Analysis

The lexical analysis phase of a compiler tokenises source code by matching character sequences against patterns defined by regular expressions, which are internally reduced to finite-automaton-based string matching (Aho, Lam, Sethi & Ullman, 2006).

6. Digital Forensics

Investigators use string searching to scan disk images for keywords, email addresses, URLs, or file signatures (magic bytes) hidden within raw binary data.

1.2 Boyer-Moore Implementation, Output Analysis and Discussion

1.2.1 How Boyer-Moore Works

Developed by Robert S. Boyer and J Strother Moore in 1977, the Boyer-Moore algorithm is widely regarded as the most efficient general-purpose string searching algorithm in practice. It achieves sub-linear average-case performance, meaning it can skip over large sections of text without even examining every character by using two preprocessing heuristics (Boyer & Moore, 1977) :

1. Bad Character (BC) Heuristic : On a mismatch, shift the pattern so the mismatched text character aligns with its rightmost occurrence in the pattern.
2. Good Suffix (GS) Heuristic : On a mismatch, shift the pattern so the suffix that already matched realigns with the next occurrence of that same suffix or matching prefix in the pattern.

The algorithm scans the pattern right-to-left on each alignment, then takes the maximum of the two heuristic shifts.

1.2.1 Python Implementation

Bad Character Table

```
def BadCharacterTable(pattern):  
  
    bad_char = {}  
    m = len(pattern)  
    # Iterate left-to-right so the last (rightmost) occurrence wins  
    for i in range(m):  
        bad_char[pattern[i]] = i    # overwrite keeps rightmost index  
  
    return bad_char
```

Step-by-step reasoning for "EXAMPLE":

Character	Index	BC Value (rightmost)
E	0	Overwritten (6)
X	1	1
A	2	2
M	3	3
P	4	4
L	5	5
E	6	6 (overwrites Index 0)

Good Suffix Table

```
def GoodSuffixTable(pattern):
    m = len(pattern)
    shift = [0] * (m + 1)
    border = [0] * (m + 1)

    # Phase 1: compute border[] right-to-left
    i = m
    j = m + 1
    border[i] = j

    while i > 0:
        # Slide right until characters match or we exhaust the border
        while j <= m and pattern[i - 1] != pattern[j - 1]:
            # If shift[j] not yet set, the current border position triggers a shift
            if shift[j] == 0:
                shift[j] = j - i
            j = border[j]
        i -= 1
        j -= 1
        border[i] = j

    # 2: handle Case 2 (prefix = suffix of matched part)
    j = border[0]
    for i in range(m + 1):
        if shift[i] == 0:
            shift[i] = j
        if i == j:
            j = border[j]

    return shift
```

Main Search Function

```
def BoyerMooreSearch(text, pattern):
    n, m = len(text), len(pattern)
    if m == 0 or m > n:
        return []

    BadChar = BadCharacterTable(pattern)
    GoodSuffix = GoodSuffixTable(pattern)
    occurrences = []
    s = 0

    while s <= n - m:
        # start from rightmost character
        j = m - 1

        while j >= 0 and pattern[j] == text[s + j]:
            # characters match, move left
            j -= 1
        if j < 0:
            occurrences.append(s)          # full match
            s += GoodSuffix[0] if GoodSuffix[0] > 0 else 1
        else:
            BadCharshift = j - BadChar.get(text[s + j], -1)
            GoodSuffixshift = GoodSuffix[j + 1]
            s += max(max(BadCharshift, GoodSuffixshift), 1)
        # always advance  $\geq 1$ 

    return occurrences
```


1.2.2 Output Analysis

Source Code : <https://colab.research.google.com/drive/1Evks66PJTJ7yMhFZR2O27JyDY0d5BXCe?usp=sharing>

Metric	Test Case 1	Test Case 2	Test Case 3	Test Case 4
Text	HERE IS A SIMPLE EXAMPLE	ABCABCABCABC	AAAAABAAAAA	GCATCGCAGAGAGTATACAGTACG
Pattern	EXAMPLE	ABC	BAB	GCAGAGAG
Length (n / m)	24 / 7	12 / 3	11 / 3	24 / 8
Matches found	1 (index 17)	4 (0, 3, 6, 9)	0 (none)	1 (index 5)
Boyer-Moore steps	5	4	8	5
Brute-force steps	18	10	9	17

Efficiency vs brute	72% fewer	60% fewer	11% fewer	71% fewer
Dominant heuristic	BC (Step 1) + GS (Step 3)	GS (post-match shift)	GS (barely, near worst-case)	BC = GS tie (Steps 2, 4)
Purpose	Show typical operation with both heuristics active	Verify multiple match reporting	Expose worst-case (low-entropy alphabet)	Bioinformatics, non-trivial GS table

Empirical Validation of Complexity: The study of the output over these four test cases clearly illustrates the theoretical efficiency of the method. In Test Case 1 and Test Case 4, the method improved the step counts by 72% and 71% compared to the brute-force upper bound. This empirically confirms the average-case time complexity of $O(n/m)$, showing that the combination of the Bad Character and Good Suffix heuristics allows the algorithm to safely skip large parts of the text and to examine fewer characters than the overall length of the text itself. However, Test Case 3 illustrates empirically that the method is susceptible to low-entropy data, degrading closer to the worst-case $O(nm)$ scenario.

1.2.3 Time and Space Complexity Analysis

Preprocessing Phase

Component	Time	Space
Bad Character Table	$O(m)$	$O(n)$
Good Suffix Table	$O(m)$	$O(m)$
Total Preprocessing	$O(m + n)$	$O(m + n)$

Where n = alphabet size.

Search Phase

Component	Time Complexity	When it occurs
Best / Average	$O(n/m)$	Large alphabets, long diverse patterns
Worst Case	$O(nm)$	Repetitive text with repetitive pattern
Best Case (no Match)	$O(n/m)$	Pattern's last char rarely appears in text

The $O(n/m)$ average case means Boyer-Moore examines only a fraction of the text characters. For a text of 1,000,000 characters and a pattern of length 10, the algorithm may only need about 100,000 comparisons in the best case, compared to 10,000,000 for brute force.

1.3 Strengths and Weaknesses of Boyer-Moore

1.3.1 Strengths of Boyer-Moore

1. **Sub-linear average performance:** The algorithm is faster than $O(n)$ on average for large alphabets, which no brute-force approach can achieve. In practice, it often makes fewer comparisons than the length of the text.
2. **No backtracking in the text:** The text cursors only move forward, which is cache-friendly for modern CPU architectures and essential for streaming applications.
3. **Scales efficiently with long patterns:** The longer the pattern, the larger the potential shifts, making the algorithm faster as patterns grow. Unlike brute force, which becomes slower

4. **Dual heuristics provide robustness:** The max of Bad Character and Good Suffix shifts guarantees that no single heuristic bounds the performance, because they compensate for their respective weakness in different situations.
5. **Practical Benchmark :** Boyer-Moore is the algorithm used in many production systems because of its empirically superior performance on real-world text.

1.3.2 Weaknesses of Boyer Moore

1. **Worst-Case $O(nm)$ performance :** On highly repetitive inputs (searching "aaa...ab" in "aaa...a"), the GS and BC shifts may both reduce to 1, leading to quadratic behaviour.
2. **Preprocessing overhead:** Computing the Good Suffix table in $O(m)$ requires a non-trivial border-array construction, making the implementation considerably more complex than KMP or brute force. This overhead may be uneconomical when the text is very short or the pattern is searched only once.
3. **High space usage for large alphabets:** The Bad Character table can consume $O(\sigma)$ space. For Unicode ($\sigma = 65,536 +$), a hash map must be used instead of a direct array, adding a small constant overhead per lookup.
4. **Right-to-left scanning is counterintuitive:** Most natural language processing assumes left-to-right reading, debugging and verifying Boyer-Moore implementations requires careful attention to the scanning direction.
5. **Less competitive on short patterns:** For patterns of length 1–3, the preprocessing cost outweighs the savings. Simpler algorithms may be faster in practice for very short patterns

2.0 Part B : Variations of the Boyer-Moore Algorithm

2.1 Introduction to Algorithm Variations

Structurally, the original Boyer-Moore (BM) algorithm is designed as a highly optimized string matching algorithm which means it accelerates search speeds by scanning patterns from right to left rather than the traditional left to right approach. It actively relies on two core heuristics which are the "Bad Character" and "Good Suffix" rules to calculate safe jump distances. As a result, the system can skip over large blocks of non matching text instead of checking every single character.

On the other hand, despite its raw processing speed, the standard Boyer-Moore algorithm operates with a high memory overhead. It requires significant preprocessing to calculate and store both shift tables before the actual search even begins. This structural bottleneck eventually led to the development of several algorithm variations designed to optimize different performance metrics including Space Complexity Reduction, Shift Optimization and Implementation Simplicity.

One primary area of optimization focuses on Space Complexity Reduction. To achieve this, some variations operate by dropping the Good Suffix rule entirely. This functions as a highly memory efficient model. Consequently, it drastically reduces the memory footprint and setup time without severely impacting the practical search speed. This specific optimization forms the foundational logic of the Boyer-Moore-Horspool variation.

Furthermore, Shift Optimization represents another critical improvement. Under this approach, other variations continuously push for faster execution by modifying the scanning window. They evaluate the character immediately after the current alignment to calculate larger and more aggressive jump distances. This shift logic forms the basis for both the Turbo Boyer Moore and Sunday variations.

Finally, achieving Implementation Simplicity is a crucial factor for developers. The standard Good Suffix rule is notoriously complex to code. Therefore, newer variations provide a cleaner and more straightforward evaluation logic. This refinement reduces the risk of logical errors while maintaining robust performance. This structural focus characterizes the Boyer Moore Horspool and Sunday variations.

2.2 Variation 1: Boyer-Moore-Horspool Algorithm

Structurally, the Boyer-Moore-Horspool algorithm operates as a simplified and highly memory efficient derivative of the original Boyer-Moore algorithm. Instead of utilizing multiple heuristics, it relies exclusively on a modified Bad Character rule. When the algorithm scans a text segment and detects a mismatch, it operates independently of the exact mismatch position. Instead, it always inspects the rightmost character of the current text window (Cheung, n.d.) . It then uses this specific rightmost character to look up the safe jump distance from a precomputed table. This mechanism ensures a continuous forward push without the need for complex suffix evaluations.

2.2.1 Key Differences Between Boyer-Moore-Horspool and Boyer-Moore Algorithm

The primary algorithmic difference between Horspool and the standard Boyer-Moore lies in heuristic dependency. The original algorithm must calculate and compare shift values from both the Bad Character and Good Suffix tables before executing a shift. In contrast, the Horspool algorithm completely drops the Good Suffix rule. Furthermore, while the standard Bad Character rule evaluates the exact character that caused the mismatch, Horspool unconditionally evaluates the rightmost character of the text window. This specific modification significantly reduces preprocessing overhead while maintaining aggressive shift distances. Consequently, it optimizes the overall space complexity to $O(\Sigma)$ as the mechanism only allocates memory for the character alphabet size rather than the pattern dependent tables required by the original model.

2.2.2 Pseudocode & Complexity Analysis

```
ALGORITHM HorspoolSearch(Text, Pattern)
INPUT: Text of length n, Pattern of length m
OUTPUT: Index of valid shifts where Pattern occurs in Text

m <- length(Pattern)
n <- length(Text)
ShiftTable <- BuildBadCharacterTable(Pattern)

i <- 0
WHILE i <= (n - m) DO
    j <- m - 1

    // Scan right to left
    WHILE (j >= 0) AND (Pattern[j] = Text[i + j]) DO
        j <- j - 1
    END WHILE

    // Check for complete match
    IF j < 0 THEN
        PRINT "Pattern found at index", i
    END IF

    // Unconditional shift using the rightmost character of the window
    i <- i + ShiftTable[Text[i + m - 1]]
END WHILE
```

When evaluating time complexity, the algorithm achieves a best case execution time of $O(n/m)$. In this mathematical model, n represents the text length and m denotes the pattern length. This optimal speed occurs when the rightmost character consistently mismatches and does not exist in the pattern. The average search case typically runs in $O(n)$. However, in the worst case scenario, the time complexity degrades significantly to $O(mn)$ (Saldana, 2018). This specific worst case execution typically happens when processing highly repetitive patterns. Furthermore, we must also consider space complexity. The algorithm operates with a space efficiency of $O(\Sigma)$. In this context, Σ represents the total size of the character alphabet. This low memory footprint is a direct result of the algorithm only needing a single and one dimensional shift table to function properly.

2.2.3 Strengths and Weaknesses

Operationally, the Boyer Moore Horspool algorithm demonstrates significant algorithm strengths when deployed within systems that process large character alphabets. This specific environment typically includes standard English text databases or extensive ASCII document processing. Under these conditions, the probability of encountering a mismatched character that does not exist in the search pattern is exceptionally high.

Consequently, the algorithm frequently achieves its maximum shift distance. Furthermore, the complete exclusion of the Good Suffix rule guarantees a much faster preprocessing phase alongside a minimal memory requirement. This specific structural advantage makes the algorithm highly suitable for real time text editors and standard search engines.

On the other hand, the algorithm exposes critical structural weaknesses when tasked with analyzing datasets built on very small alphabets. This performance limitation becomes highly apparent in binary data analysis or DNA sequence mapping. In these specialized computing scenarios, characters repeat constantly. As a direct result, the Bad Character table generates extremely short shift distances. The algorithm lacks the Good Suffix rule to act as an alternative heuristic fallback. Therefore, the algorithm is forced into performing nearly continuous character comparisons. This theoretical limitation inevitably degrades the overall algorithm performance toward the worst case execution time.

2.3 Variation 2: Turbo Boyer Moore Algorithm

The Turbo Boyer Moore algorithm functions as a state aware enhancement of the standard Boyer Moore model. It improves search efficiency by retaining knowledge of the previously matched suffix within its internal memory state. When the algorithm identifies a character mismatch after a partial pattern match, it registers the length of the successfully matched suffix. During the subsequent pattern alignment, the search mechanism recalls this stored value to calculate a ‘turbo shift’. This refined logic ensures that the algorithm avoids reevaluating text segments that have already been verified. Consequently, it transforms a stateless evaluation process into a sophisticated and memory integrated search operation.

2.3.1 Key Differences Between Turbo Boyer-Moore Algorithm and Boyer-Moore Algorithm

The primary algorithmic difference between the Turbo variation and the standard Boyer Moore model involves data retention during the shift phase. The standard model operates as a stateless mechanism which purges all information regarding matched characters immediately upon detecting a mismatch. In contrast, the Turbo Boyer Moore algorithm retains the length of the previously matched suffix. This persistent state allows the algorithm to perform a specialized shift calculation that bypasses redundant character comparisons. Furthermore, the Turbo variation enables larger shift distances compared to the original model by leveraging the memory of past matches to optimize its movement. This mechanism effectively reduces the total number of comparisons, thereby improving the efficiency of the search process in complex data patterns.

2.3.2 Pseudocode & Complexity Analysis

```
ALGORITHM TurboBoyerMooreSearch(Text, Pattern)
INPUT: Text of length n, Pattern of length m
OUTPUT: Index of valid shifts where Pattern occurs in Text

m <- length(Pattern)
n <- length(Text)
BadCharTable <- BuildBadCharacterTable(Pattern)
GoodSuffixTable <- BuildGoodSuffixTable(Pattern)

// Memory state to track length of previously matched suffix
Memory <- 0

i <- 0
WHILE i <= (n - m) DO
    j <- m - 1

    // Scan right to left
    WHILE (j >= 0) AND (Pattern[j] = Text[i + j]) DO
        j <- j - 1

        // Skip the matched memory block to guarantee O(n) operations
        IF j = (m - 1 - Memory) THEN
            j <- j - Memory
        END IF
    END WHILE

    // If the entire pattern matched
    IF j < 0 THEN
        PRINT "Pattern found at index", i
        Shift <- GoodSuffixTable[0]
        Memory <- m - Shift
        i <- i + Shift
    ELSE
        // 1. Calculate standard heuristic shifts
        BadCharShift <- BadCharTable[Text[i + j]] - (m - 1 - j)
        GoodSuffixShift <- GoodSuffixTable[j]

        // 2. The crucial Turbo Shift calculation
        TurboShift <- Memory - (m - 1 - j)

        // 3. Determine the maximum safe jump
        Shift <- MAX(BadCharShift, GoodSuffixShift, TurboShift)

        // 4. Update the memory state for the next alignment
        IF Shift = GoodSuffixShift THEN
            Memory <- MIN(m - Shift, m - 1 - j)
        ELSE
            IF TurboShift < GoodSuffixShift THEN
                Shift <- MAX(Shift, Memory + 1)
            END IF

            // Reset memory state if non-suffix heuristic dominates
            Memory <- 0
        END IF

        i <- i + Shift
    END IF
END WHILE
```

In terms of computational complexity, the Turbo Boyer Moore algorithm demonstrates distinct performance metrics across different operational scenarios. Under optimal conditions, the search mechanism achieves a best case time complexity of $O(n/m)$ where n represents the total text length and m denotes the pattern length. The average case search execution typically scales linearly to maintain a time complexity of $O(n)$.

However, unlike standard algorithmic derivatives, the Turbo Boyer-Moore mechanism maintains a strict linear worst case time complexity of $O(n)$. This linear performance happens because the algorithm utilizes an internal memory state which directly guarantees this mathematical upper bound. The mechanism remembers previous matches and completely avoids reevaluating previously verified text segments during repetitive mismatch scenarios. Alongside time efficiency, the structural design of the algorithm requires a specific memory overhead that defines its space complexity. Regarding memory consumption, the mechanism operates with a space complexity of $O(m + \Sigma)$ where Σ represents the total size of the character alphabet. This specific memory footprint is mandatory to construct both the Bad Character and the Good Suffix tables. The algorithm strictly requires these complete tables to execute its memory integrated shift logic.

2.3.3 Strengths & Weaknesses

The Turbo Boyer Moore algorithm demonstrates distinct strengths when deployed in extensive text processing applications that feature large alphabet sizes and low character redundancy such as standard natural language corpuses. Under these conditions, the search mechanism effectively leverages its memory of previously matched suffixes to bypass redundant character evaluations. For instance, when a character mismatch occurs after a substantial partial match, the algorithm recalls the verified suffix length to execute a highly maximized shift distance. This specific design allows the mechanism to outperform the standard Boyer Moore model as it completely eliminates the need to reverify characters that have already been matched in a prior alignment window. Consequently, this capability ensures high efficiency in deep text analysis where execution speed is a primary requirement.

Conversely, the algorithm presents notable performance constraints when applied to datasets characterized by small alphabets or highly repetitive sequences. In specialized data processing contexts such as binary stream tracking or genomic sequence analysis, character mismatches frequently occur at the very first position of the pattern window. This operational behavior prevents the algorithm from establishing a matched suffix in its memory state. Therefore, the potential to execute a specialized turbo shift becomes heavily restricted by these constant early mismatch points. This limitation results in diminishing computational returns as the execution overhead required to manage the additional memory tracking variables yields no practical speed advantage over simpler models like the Horspool variation. Furthermore, the algorithm demands higher memory allocation to maintain both the Bad Character and Good Suffix tables simultaneously which further restricts its suitability for memory constrained execution scenarios.

2.4 Variation 3: Sunday Algorithm

The Sunday algorithm or frequently referred to as the Quick Search algorithm which functions as a highly aggressive string matching mechanism. Unlike previous models, it modifies the standard evaluation process through a specific lookahead approach. When a character mismatch occurs, the algorithm completely bypasses the characters within the current alignment window. Instead, it immediately inspects the single text character located just outside the right boundary of the search pattern (ECWEB, 2025). The algorithm then utilizes this specific lookahead character to retrieve the appropriate jump distance from a precomputed shift array. Ultimately, this forward looking logic enables the mechanism to execute much larger shift distances. This specific design choice significantly accelerates the overall search operation.

2.4.1 Key Differences Between Sunday Algorithm and Boyer-Moore Algorithm

The primary algorithmic difference between the Quick Search approach and the standard Boyer Moore model lies in the reference point used for shift calculations. The original Boyer Moore and the Horspool variation evaluate characters strictly within the current pattern window to determine their next movement. In direct contrast, the Sunday algorithm takes a completely different approach. It evaluates a character located entirely outside the current alignment window. Specifically, the algorithm directly targets the text character at position $\text{text}[i+m]$ where m represents the pattern length. Furthermore, the Sunday approach simplifies its overall logic by dropping the Good Suffix rule entirely. It relies solely on a modified Bad Character table to dictate every single shift operation. Consequently, this design optimizes its space complexity to $O(\Sigma)$ as the mechanism only allocates memory for the character alphabet size.

2.4.2 Pseudocode & Complexity Analysis

```
ALGORITHM SundayQuickSearch(Text, Pattern)
INPUT: Text of length n, Pattern of length m
OUTPUT: Index of valid shifts where Pattern occurs in Text

m <- length(Pattern)
n <- length(Text)

// Build shift table for the entire alphabet
ShiftTable <- BuildQuickSearchTable(Pattern)

i <- 0
WHILE i <= (n - m) DO
    j <- 0

    // Forward character comparison
    WHILE (j < m) AND (Pattern[j] = Text[i + j]) DO
        j <- j + 1
    END WHILE

    // Check for complete match
    IF j = m THEN
        PRINT "Pattern found at index", i
    END IF

    // Prevent index out of bounds before shifting
    IF (i + m) < n THEN
        // Shift based on the character strictly after the window
        i <- i + ShiftTable[Text[i + m]]
    ELSE
        // Terminate loop safely at the end of text
        BREAK
    END IF
END WHILE
```

Regarding computational complexity, the Sunday algorithm demonstrates performance metrics closely aligned with other simplified matching models. Specifically, the mechanism achieves a best case time complexity of $O(n/m)$ under optimal execution conditions. This mathematical optimum occurs when the character immediately following the window consistently mismatches and does not exist in the pattern. For the average case, the operation scales linearly to maintain a time complexity of $O(n)$. Conversely, the worst case performance degrades to $O(mn)$. This theoretical limitation typically surfaces when the mechanism processes highly repetitive data structures. Finally, the algorithm maintains a minimal space complexity of $O(\Sigma)$. This high memory efficiency is a direct result of the design only requiring a single and one dimensional shift array for the character alphabet.

2.4.3 Strengths & Weaknesses

Operationally, the Sunday algorithm demonstrates distinct strengths when processing short search patterns across large character alphabets. In practical computing applications such as basic text editors or standard string extraction utilities, inspecting the character immediately after the alignment window frequently yields the maximum possible shift distance. This specific algorithmic behavior translates to extremely fast practical search speeds. Furthermore, the reliance on a single shift table ensures rapid preprocessing times and minimal memory consumption.

Conversely, the algorithm manifests notable performance constraints when tasked with analyzing data derived from small alphabets. In analytical scenarios involving binary stream processing or genomic sequence mapping, the probability of the lookahead character existing within the search pattern is exceptionally high. As a direct consequence, the calculated shift distances become remarkably short. The absence of an alternative heuristic fallback results in the mechanism executing continuous and redundant character comparisons. Therefore, the overall search performance degrades significantly when processing highly repetitive data corpuses.

2.5 Comparative Analysis

Algorithm	Time Complexity	Space Complexity	Primary Strengths	Key Weaknesses
Boyer Moore	Best Case: $O(n/m)$ Average Case: $O(n)$ Worst Case: $O(mn)$	$O(m + \Sigma)$	Utilizes both Bad Character and Good Suffix rules to calculate safe jumps. It efficiently skips large blocks of non matching text.	Requires significant memory overhead. The mechanism involves a complex and lengthy preprocessing phase.
Boyer Moore Horspool	Best Case: $O(n/m)$ Average Case: $O(n)$ Worst Case: $O(mn)$	$O(\Sigma)$	Relies exclusively on the rightmost character within the window. This specific design ensures rapid preprocessing and minimal memory consumption.	Lacks an alternative heuristic fallback. The search performance degrades significantly when processing datasets with small alphabets.
Turbo Boyer Moore	Best Case: $O(n/m)$ Average Case: $O(n)$	$O(m + \Sigma)$	Integrates a memory state to bypass previously matched suffixes.	Demands higher memory allocation to maintain multiple tables simultaneously.

	Worst Case: $O(n)$		It outperforms standard models during deep text analysis with low character redundancy.	It yields diminishing returns in highly repetitive sequences.
Sunday (Quick Search)	Best Case: $O(n/m)$ Average Case: $O(n)$ Worst Case: $O(mn)$	$O(\Sigma)$	Inspects the specific lookahead character located immediately after the search window. It executes maximum shift distances for short patterns across large alphabets.	Calculates remarkably short shift distances when analyzing small alphabets. The mechanism executes continuous redundant comparisons during early mismatch scenarios.

2.6 Performance Considerations

To determine the most suitable algorithm, an evaluation of the specific text size and the expected pattern length is necessary. Each mechanism offers distinct advantages under different structural conditions including the analysis of short, medium and long search patterns. The Boyer-Moore-Horspool variation is highly suitable for medium length patterns within standard text files. The mechanism drops the Good Suffix rule to minimize memory consumption. It relies solely on the rightmost character to calculate safe shift distances. Alternatively, the Sunday algorithm performs exceptionally well when searching for short patterns across massive text sizes. The search logic inspects the character located immediately outside the current window. This lookahead approach forces maximum jump distances to accelerate the overall search operation.

On the other hand, the Turbo Boyer-Moore variation serves as the optimal choice for long search patterns and highly repetitive text structures. The algorithm utilizes an internal memory state. It remembers previously matched suffixes to avoid redundant character comparisons. This specific logic successfully maintains a linear worst case speed of $O(n)$ during deep text analysis. Finally, the original Boyer-Moore algorithm provides a historical baseline for long patterns. However, Horspool or Turbo algorithms serve as more efficient practical alternatives for modern applications since the original model requires a high memory overhead to construct dual shift tables.

3.0 Part C : Comparison with the KMP Algorithm

3.1 Introduction

The KMP algorithm is an extremely efficient linear-time string search algorithm that searches for a given pattern in the main string (GeeksforGeeks, 2026) . The KMP was invented by James H. Morris, Vaughan Pratt and Donald Knuth. It surpasses the brute force method because of its better complexity. While the brute force technique requires $O(n \times m)$ complexity in the worst case because of redundant character comparison, KMP ensures a worst-case complexity of $O(n + m)$. Here, n represents the length of the main string and m denotes the length of the pattern. Thus, it becomes easier and more efficient to search when repeating patterns occur in the text string.

3.1.1 Primary Mechanism: The Prefix Table (LPS Array)

The basic idea behind the efficiency of the KMP algorithm is that the pattern contains enough information about itself to determine how far ahead the search should continue in case of a mismatch. This is done through preprocessing of the pattern into a special partial match table, also called the Longest Prefix Suffix or LPS table.

Term	Definition
Prefix	The term substring begins from the very first character of the pattern.
Suffix	The term substring ends at the last character of the pattern.
Proper Prefix/Suffix	A prefix/suffix whose size is less than the entire string.

This process works according to the following two principles:

Purpose: The LPS array will examine the sequence and calculate the length of the longest proper prefix that is identical to a proper suffix for each sub-sequence in the text up until each letter.

Operation: Through the pre-computation of the pattern, a table of fallback values will be generated. In case of a mismatch between the string and the pattern, the LPS array will provide the distance by which the pattern index should move backwards. The text index will never move backwards.

3.1.2 Illustration with an Example

Take into consideration the pattern = A B A B C A.

After the preprocessing step, the LPS array takes the following form:

Index	0	1	2	3	4	5
Pattern P[i]	A	B	A	B	C	A
LPS Array	0	0	1	2	0	1

Let us assume that the string to be searched contains A B A B X ... and the pattern is A B A B C A. In this case, the function of the shift can be explained as:

1. The Match: Here, matching of the characters takes place till index 3 (A B A B).
2. The Mismatch: There is a mismatch at index 4 (There is an X in the text and not a C).
3. The Table Lookup: Rather than shifting the pattern one position to the right and starting matching all over again, LPS of the character prior to the mismatched one is determined, which is equal to 2 in this case because $LPS[3] = 2$. This denotes that the longest prefix which is also a suffix of the successful match (A B A B) has a length of 2.
4. The Shift: The pattern automatically shifts to make the prefix (A B) align with its suffix (A B) in the text and the matching continues from the second index onwards of the pattern.

3.1.3 KMP Algorithm Pseudocode

ALGORITHM KMPSearch(Text, Pattern)

INPUT: Text of length n, Pattern of length m

OUTPUT: Starting indices where Pattern occurs in Text


```

n <- length(Text)
m <- length(Pattern)

// PHASE 1: PREPROCESSING (COMPUTE LPS ARRAY)
allocate lps array of size m initialized to 0
length_prev_lps <- 0
i_lps <- 1

WHILE i_lps < m DO
  IF Pattern[i_lps] = Pattern[length_prev_lps] THEN
    length_prev_lps <- length_prev_lps + 1
    lps[i_lps] <- length_prev_lps
    i_lps <- i_lps + 1
  ELSE
    IF length_prev_lps != 0 THEN
      // Revert to the prior longest prefix suffix
      length_prev_lps <- lps[length_prev_lps - 1]
    ELSE
      lps[i_lps] <- 0
      i_lps <- i_lps + 1
    END IF
  END IF
END WHILE

// PHASE 2: SEARCHING PHASE
i <- 0 // Index for Text
j <- 0 // Index for Pattern

WHILE i < n DO
  IF Pattern[j] = Text[i] THEN
    i <- i + 1
    j <- j + 1
  END IF

  IF j = m THEN
    PRINT "Pattern found at index", i - j
    // Retrieve the next matching position from the LPS array
    j <- lps[j - 1]
  ELSE
    IF i < n AND Pattern[j] != Text[i] THEN
      // Handle mismatch after j successful matches
      IF j != 0 THEN
        // Bypass characters already validated by the LPS array
        j <- lps[j - 1]
      ELSE
        i <- i + 1
      END IF
    END IF
  END IF
END WHILE

```

3.2 Comparison of the Boyer-Moore and KMP Algorithms

The KMP and Boyer-Moore algorithms can be considered two entirely different techniques to improve the efficiency of string matching. While KMP is based on eliminating backtracking of the text pointer (left to right scanning), Boyer-Moore is based on jumping as much as possible over the scanned patterns (right to left scanning).

3.2.1 Comparison Summary

Comparison Criteria	Knuth-Morris-Pratt (KMP)	Boyer-Moore (BM)
Best-Case Time Complexity	$O(n)$	$O(n/m)$ (Sublinear)
Worst-Case Time Complexity	$O(n + m)$	$O(mn)$
Space Complexity	$O(m)$	$O(m + \Sigma)$
Primary Strengths	- Guaranteeing strict time constraints in a worst-case scenario gives mathematical reliability. - Reads input linearly without revisiting anything previously seen, thus becoming very appropriate for stream processing.	- Very fast due to character jumps being quite high in practical scenarios. - Speeds up considerably with an increase in the pattern's length. - Well suited for systems dealing with a very large alphabet of characters (such as normal English text).
Primary Weaknesses	- Slower than its average case because it analyses more characters. - Cannot jump ahead by much because of its lack of structure.	- Needs substantial memory space to build and keep two lookup tables. - Is not appropriate for searching in streams of characters, since it scans from right to left. - Has a complicated preprocessing step.

3.2.2 Pseudocode Illustration of Complexity Differences

This difference comes from the way their inner loop works.

Knuth-Morris-Pratt (Left-to-Right Scan):

ALGORITHM KMPComplexityLoop(Text, Pattern)

INPUT: Text of length n , Pattern of length m

OUTPUT: Linear character matching execution

```
lps <- ComputeLPS(Pattern)
i <- 0
j <- 0

WHILE i < n DO
  IF Pattern[j] = Text[i] THEN
    i <- i + 1      // Text pointer strictly increments forward
    j <- j + 1
  END IF

  IF j = m THEN
    j <- lps[j - 1]
  ELSE
    IF i < n AND Pattern[j] != Text[i] THEN
      IF j != 0 THEN
        j <- lps[j - 1] // Pattern shifts without text pointer regression
      ELSE
        i <- i + 1
      END IF
    END IF
  END IF
END WHILE
```

The KMP algorithm always takes $O(n)$ time since its internal loop causes the text index 'i' to increment or remain constant.

Boyer-Moore (Right-to-Left Scan):

ALGORITHM BoyerMooreComplexityLoop(Text, Pattern)

INPUT: Text of length n , Pattern of length m

OUTPUT: Shift-based character matching execution

```

bad_char <- BuildBadCharTable(Pattern)
good_suffix <- BuildGoodSuffixTable(Pattern)
shift <- 0

WHILE shift <= (n - m) DO
  j <- m - 1

  // Right-to-left inner character comparison loop
  WHILE j >= 0 AND Pattern[j] = Text[shift + j] DO
    j <- j - 1
  END WHILE

  IF j < 0 THEN
    PRINT "Pattern found at index", shift
    shift <- shift + good_suffix[0]
  ELSE
    // Shift distance is determined by the maximum heuristic alignment jump
    shift <- shift + max(j - bad_char[Text[shift + j]], good_suffix[j])
  END IF
END WHILE

```

Boyer-Moore is able to have a very high performance of $O(n/m)$ in the best cases, but in worst cases, where the text contains lots of repetitions, the inner loop can iterate m times in nearly all iterations, making this algorithm slow again to $O(mn)$.

3.2.3 Summary of Comparison

The choice between KMP and BM algorithms is determined by the peculiarities of the data environment. The BM algorithm can be considered the most suitable for performing generalized searches in large alphabets, providing outstanding sub-linear time through ignoring unnecessary characters' comparisons. Nevertheless, the BM algorithm requires more space due to its two shift tables and may experience decreased efficiency while working with repeating strings.

On the contrary, the KMP algorithm is highly stable and linear-time efficient. In addition, it can be applied in cases when there is no extra memory available, in the case of a small alphabet and if the reversal of the text pointer cannot be performed because of continuous data streams. Both algorithms solve the problem of inefficiency of brute force search using entirely different approaches to data scan KMP uses mathematical regularity and BM execution speed.

3.3 Scenarios: Algorithm Suitability

Whether one should use KMP or Boyer-Moore largely depends on the situation at hand.

Scenario 1: Creation of Basic Text Editor (such as MS Word)

- Best Approach: Boyer-Moore
- Reasons: The nature of a text editor means that it will work on texts written in human languages. These contain big alphabets (English letters, characters, numbers). With big alphabets, the chance of the existence of a certain text character in a pattern is very low. This makes it possible for Boyer-Moore to always reach its best time complexity of $O(n/m)$ since it skips large chunks of texts. Using KMP will take longer due to lack of capacity for executing those jumps.

Scenario 2: DNA Sequence Alignment

- Best Approach: Knuth-Morris-Pratt
- Reasons: DNA Sequences consist of a very small set of characters ($\Sigma = 4$: A,C,G,T). In situations where there is small alphabet size and patterns tend to repeat themselves, Boyer-Moore becomes almost useless because its shifts become tiny leading to almost continuous comparison.

Scenario 3: Real Time Inspect Network Packets Streaming Data

- Best Approach: Knuth-Morris-Pratt (KMP)
- Reasons: Firewalls must be able to process packets of data precisely when they come into the system (in streaming format). Boyer-Moore needs to hop between positions in the text window while comparing characters in reverse order; therefore, buffering of lots of data is necessary prior to the search operation. Unlike Boyer-Moore, KMP processes text in one direction (left to right), never backtracking through the text stream.

4.0 References

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2013). *Compilers: Principles, techniques, and tools: Pearson new international edition*. Pearson Higher Ed.
- Boyer, R. S., & Moore, J. S. (1977). *A fast string searching algorithm*. Communications of the ACM.
- Cheung. (n.d.). *The Boyer-Moore-Horspool algorithm*. The Department of Computer Science, Emory University.
<https://www.cs.emory.edu/~cheung/Courses/253/Syllabus/Text/Matching-Boyer-Moore2.html>
- Experts compare KMP BM and Sunday Stringmatching algorithms. (2025, November 19). ecer Website Builder.
https://ecweb.ecer.com/topic/en/detail-297965-experts_compare_kmp_bm_and_sunday_stringmatching_algorithms.html
- GeeksforGeeks. (2026, March 27). *KMP algorithm for pattern searching*.
<https://www.geeksforgeeks.org/dsa/kmp-algorithm-for-pattern-searching/>
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.
- Saldaña. (2018, November 6). *The Boyer-Moore-Horspool Algorithm*. Medium.
<https://medium.com/@fsaldana/the-boyer-moore-horspool-algorithm-51b785afde67>

Grading Rubric

Part	Criteria	CLO	Poor (1–2 marks)	Average (3–4 marks)	Good (5–7 marks)	Excellent (8–10 marks)	Weight
Part A	1. Explanation of string matching and applications	CLO2	Incomplete, inaccurate, or unclear explanation of string matching. Applications are missing or irrelevant.	Provides a basic explanation of string matching with limited or partially relevant applications.	Clearly explains string matching and provides relevant application examples.	Provides a comprehensive and precise explanation of string matching with multiple well-chosen real-world applications.	8
Part A	2. Implementation of Boyer-Moore algorithm: correct logic, code structure, comments, output analysis, and discussion of strengths/weaknesses	CLO2	Implementation contains major logic errors or is incomplete. Code structure, comments, output analysis, and discussion of strengths/weaknesses are weak or missing.	Implementation is partially correct with some logical issues. Code structure and comments are limited; output analysis and discussion are superficial.	Implementation is mostly correct and organized. Comments are adequate, and output analysis with discussion of strengths/weaknesses is reasonably explained.	Implementation is fully correct, well-structured, and clearly commented. Output analysis is accurate, with insightful discussion of strengths and weaknesses.	25
Part B	3. Explanation of at least three Boyer-Moore variations, e.g., Boyer-Moore-Horspool, Turbo Boyer-Moore, and Sunday/Quick Search algorithm	CLO2	Fewer than three variations are explained, or explanations are incorrect, vague, or missing.	Identifies at least three variations, but explanations are basic, incomplete, or lack clarity.	Clearly explains at least three variations and describes their main operational differences from the original Boyer-Moore algorithm.	Provides detailed, accurate, and well-organized explanations of at least three variations, supported by suitable examples, illustrations, or comparison points.	13
Part B	4. Comparison of the original Boyer-Moore algorithm with at least three variations: time complexity, space complexity, strengths, weaknesses, conclusion, and pseudocode to illustrate complexity	CLO3	Comparison is incomplete, inaccurate, or covers fewer than three variations. Time/space complexity, strengths/weaknesses, conclusion, and pseudocode are missing or poorly presented.	Provides a basic comparison involving at least three variations, but analysis of time/space complexity, strengths, weaknesses, and conclusion is limited. Pseudocode is included but does not clearly support	Presents a mostly accurate comparison of the original Boyer-Moore algorithm with at least three variations, with reasonable discussion of time/space complexity, strengths, weaknesses, and conclusion. Relevant pseudocode is provided and partially supports the complexity analysis.	Delivers a thorough and insightful comparison of the original Boyer-Moore algorithm with at least three variations, with accurate time and space complexity analysis, well-justified strengths and weaknesses, and a strong conclusion.	20

				the complexity discussion.		Appropriate pseudocode clearly illustrates and supports the complexity discussion.	
Part C	5. Explanation of KMP algorithm	CLO2	Explanation is incorrect, confusing, or incomplete.	Provides a partial explanation of the KMP algorithm with limited clarity.	Clearly explains the KMP algorithm with mostly accurate details.	Provides a detailed and accurate explanation of KMP, including its main mechanism, supported by a relevant example, illustration, or pseudocode.	14
Part C	6. Comparison with KMP: time complexity, space complexity, strengths, weaknesses, summary, and pseudocode to illustrate complexity	CLO3	Comparison is incomplete, inaccurate, or lacks meaningful reasoning. Time/space complexity, strengths/weaknesses, summary, and pseudocode are missing or poorly presented.	Provides a basic comparison with minimal discussion of time/space complexity, strengths, weaknesses, and summary. Pseudocode is included but does not clearly support the complexity discussion.	Presents an accurate comparison with reasonable discussion of time/space complexity, strengths, weaknesses, and summary. Relevant pseudocode is provided and partially supports the complexity analysis.	Provides a complete, clear, and insightful comparison with sound time and space complexity analysis, justified strengths and weaknesses, and a well-supported summary. Appropriate pseudocode clearly illustrates and supports the complexity discussion.	13
Part C	7. Scenarios where one algorithm is preferable	CLO2	Scenarios are missing, inaccurate, or poorly justified.	Identifies one or a few scenarios with weak or general justification.	Provides suitable scenarios with some explanation of algorithm suitability.	Presents multiple well-justified scenarios that clearly explain when one algorithm is preferable over another.	7
	Total						100